ConcertSafe

Kursinio darbo I dalis: projektavimo dokumentas

1. Problema ir idėja

Sistema vienu sakiniu: ConcertSafe – tai koncertų bilietų rezervavimo modulis, skirtas saugiai valdyti vienalaikes naudotojų užklausas ir užtikrinti, kad tas pats bilietas ar vieta nebūtų rezervuoti keliems vartotojams.

Problema ir dabartinis procesas: Bilietų rezervavimo sistemose didelės paklausos metu daug naudotojų gali beveik tuo pačiu metu pasirinkti tas pačias vietas. Tokiu atveju sistema turi ne tik parodyti, ar vieta laisva, bet ir teisingai apdoroti vienalaikes rezervavimo užklausas. Netinkamai įgyvendinus rezervavimo logiką gali atsirasti lenktynių sąlyga: du naudotojai patikrina, kad vieta laisva, ir abu gauna patvirtinimą ją rezervuoti.

Nauda: Sistema turi užtikrinti, kad rezervavimo rezultatas būtų teisingas ir nuspėjamas net tuo atveju, kai keli naudotojai vienu metu bando rezervuoti tą pačią vietą. Naudotojui turi būti aiškiai grąžinamas arba sėkmingos rezervacijos rezultatas, arba konflikto pranešimas, kai vietą jau rezervavo kitas naudotojas. Sistemos kūrimo požiūriu modulis leis praktiškai išbandyti ir įvertinti skirtingus vienalaikių operacijų valdymo sprendimus.

Naudotojai: Pagrindinis naudotojas – bilietą rezervuojantis klientas. Jis gali pasirinkti koncertą, pasirinkti vieną ar kelias vietas ir pateikti rezervavimo užklausą. Sistemoje taip pat numatomi du iš anksto paruošti klientai (tenants), atstovaujantys skirtingoms organizacijoms, kurios turi savo koncertus, vietas ir rezervacijas.

Prielaidos: Daroma prielaida, kad mokėjimas už bilietą šiame projekte nevykdomas realiai ir nėra integruojamas su mokėjimo paslaugų teikėju. Daroma prielaida, kad koncertai ir vietos sistemoje sukuriami iš anksto. Daroma prielaida, kad egzistuoja bent du iš anksto paruošti klientai, o jų registracijos ir valdymo funkcionalumas nereikalingas. Viena vieta vienu metu gali turėti tik vieną aktyvią rezervaciją. Rezervacija laikoma galiojančia tol, kol ji patvirtinama arba pasibaigia jos galiojimo laikas; rezervacijos galiojimo pabaigos mechanizmas gali būti įgyvendintas vėlesniame etape. Pagrindinis projekto tikslas yra rezervavimo operacijos teisingumas vienalaikių užklausų metu, o ne pilnos komercinės bilietų pardavimo platformos sukūrimas.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Koncerto vietų peržiūra	Pasirinkti koncertą ir gauti jo vietų bei jų būsenų sąrašą	Pagalbinė funkcija
Bilietų rezervavimas	Pateikti vietų rezervavimo užklausą ir gauti rezervacijos rezultatą	**Pagrindinis modulis**
Rezervacijos atšaukimas	Atšaukti savo rezervaciją ir grąžinti vietas į laisvų vietų būseną	Pagalbinė funkcija
Kliento duomenų atskyrimas	Veikti pasirinkto kliento kontekste ir pasiekti tik jam priklausančius koncertus bei rezervacijas	Pagalbinė funkcija, reikalinga pagrindiniam moduliui

Į kursinio darbo apimtį neįeina: Pilnas koncertų portalas, naudotojų savarankiška registracija, autentifikacija, išsami rolių ir teisių sistema, realūs mokėjimai, bilietų generavimas, el. laiškų siuntimas, koncertų paieška pagal išorinius šaltinius, rekomendacijų sistema, mobili aplikacija, debesijos diegimas ir sudėtinga administravimo sąsaja. Trumpai tariant, nebus siekiama atkartoti viso komercinės bilietų prekybos platformos funkcionalumo.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Bilietų rezervavimo ir vienalaikių operacijų valdymo modulis. Modulio atsakomybė – priimti rezervavimo užklausą, patikrinti verslo taisykles, saugiai pakeisti vietų būseną ir grąžinti aiškų rezultatą net tuo atveju, kai kelios užklausos vykdomos vienu metu.

Logika, kurią reikės projektuoti ir testuoti: Modulio esmę sudaro vietų prieinamumo tikrinimas, kliento ir koncerto priklausomybės tikrinimas, vienalaikių rezervavimo operacijų valdymas ir konflikto grąžinimas. Bus siekiama užtikrinti, kad rezervuojant tą pačią vietą vienu metu keliems klientams tik viena operacija galėtų būti sėkminga.

Įvestis: kliento identifikatorius, pvz., tenant-a;

koncerto identifikatorius, pvz., concert-101;

naudotojo identifikatorius, pvz., customer-25;

vienos ar kelių pasirinktų vietų identifikatoriai, pvz., A1, A2.

Išvestis: Sėkmės atveju sistema grąžins rezervacijos identifikatorių, rezervuotų vietų sąrašą ir rezervacijos būseną.

Pavyzdys:

```
{
  "reservationId": "res-501",
  "status": "RESERVED",
  "seatIds": ["A1", "A2"]
}
```

Konflikto atveju sistema grąžins, kad bent viena pasirinkta vieta jau rezervuota.

Pavyzdys:

```
{
  "status": "CONFLICT",
  "message": "Seat A1 is no longer available."
}
```

Veikimo eiga:

1. Priimama rezervavimo užklausa su kliento, koncerto, naudotojo ir vietų duomenimis.
2. Patikrinama, ar nurodytas koncertas ir vietos priklauso pasirinktam klientui.
3. Patikrinama, ar visos pasirinktos vietos egzistuoja ir yra laisvos.
4. Rezervavimo operacija atliekama atominiu būdu, kad kelios vienalaikės užklausos negalėtų sėkmingai rezervuoti tos pačios vietos.
5. Sėkmės atveju sukuriama rezervacija ir atnaujinamos vietų būsenos.
6. Konflikto arba klaidos atveju pakeitimai neatliekami ir naudotojui grąžinamas aiškus rezultatas.

Taisyklės arba sprendimo žingsniai

1. Vieta gali būti rezervuota tik tuo atveju, jei ji priklauso nurodytam koncertui ir šio koncerto kliento kontekstui. Bandymas pasiekti kito kliento vietą turi būti atmestas.
2. Viena vieta gali turėti tik vieną aktyvią rezervaciją. Jei dvi ar daugiau vienalaikių užklausų bando rezervuoti tą pačią vietą, sėkminga gali būti tik viena iš jų, o kitos turi gauti konflikto rezultatą.
3. Rezervacija sukuriama tik tada, kai visos užklausoje nurodytos vietos gali būti rezervuotos. Jei bent viena pasirinkta vieta nebegalima, visa rezervavimo operacija atmetama ir neturi likti dalinės rezervacijos.
4. Kito kliento duomenys negali būti keičiami ar skaitomi. Net jei naudotojas žino kito kliento koncerto ar rezervacijos identifikatorių, sistema turi atmesti tokią užklausą.

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	Klientui tenant-a priklauso koncertas concert-101. Vietos A1 ir A2 yra laisvos. Naudotojas customer-25 prašo A1 ir A2.	Pateikiama rezervavimo užklausa	Rezervacija sukuriama, grąžinama sėkmės būsena, o A1 ir A2 tampa rezervuotos.
Ribinis atvejis arba konfliktas	Vietą A1 tuo pačiu metu bando rezervuoti 20 užklausų. A1 pradžioje yra laisva.	Vienu metu paleidžiamos 20 rezervavimo užklausų dėl A1.	Lygiai viena užklausa baigiasi sėkmingai, 19 gauna konflikto rezultatą. Vietai A1 egzistuoja tik viena aktyvi rezervacija.
Klaida arba neįmanomas rezultatas	Vietos A1 ir A2 yra laisvos, tačiau A3 neegzistuoja arba jau rezervuota. Naudotojas prašo A1, A2, A3.	Pateikiama kelių vietų rezervavimo užklausa	Rezervacija neatliekama nė vienai vietai: A1 ir A2 lieka laisvos, o sistema grąžina aiškų klaidos arba konflikto rezultatą.
Klientų izoliavimas	concert-101 priklauso tenant-a, o tenant-b neturi teisės jo valdyti.	tenant-b bando rezervuoti vietą iš concert-101	Užklausa atmetama; tenant-b negauna tenant-a duomenų ir negali pakeisti jo vietų ar rezervacijų.

Jei modulis naudoja AI: Netaikoma.

4. Kokybės atributas

Pasirinktas atributas: duomenų vientisumas ir teisingumas vienalaikių operacijų metu.

Kodėl svarbus šiai sistemai: Bilietų rezervavimo sistemoje svarbiausia, kad vienas konkretus bilietas arba vieta nebūtų paskirti keliems naudotojams. Didžiausia pasirinkto modulio rizika yra lenktynių sąlyga, kai keli klientai vienu metu pateikia rezervavimo užklausas. Todėl vien tik funkcinio scenarijaus patikrinimo nepakanka – būtina patikrinti sistemos elgesį esant vienalaikėms operacijoms.

Tikrinimo scenarijus ir sąlygos: Koncerto concert-101 vieta A1 pradžioje yra laisva. Vienu metu paleidžiama 100 rezervavimo užklausų, iš kurių visos bando rezervuoti A1. Testas vykdomas lokaliai dokumentuotoje kūrimo aplinkoje.

Sėkmės kriterijus: Lygiai 1 užklausa baigiasi sėkminga rezervacija, 99 užklausos grąžina konflikto rezultatą, duomenų bazėje egzistuoja tik viena aktyvi A1 rezervacija, vieta neturi dviejų skirtingų savininkų, rezervacijų skaičius nesumažina inventoriaus iki neigiamos reikšmės.

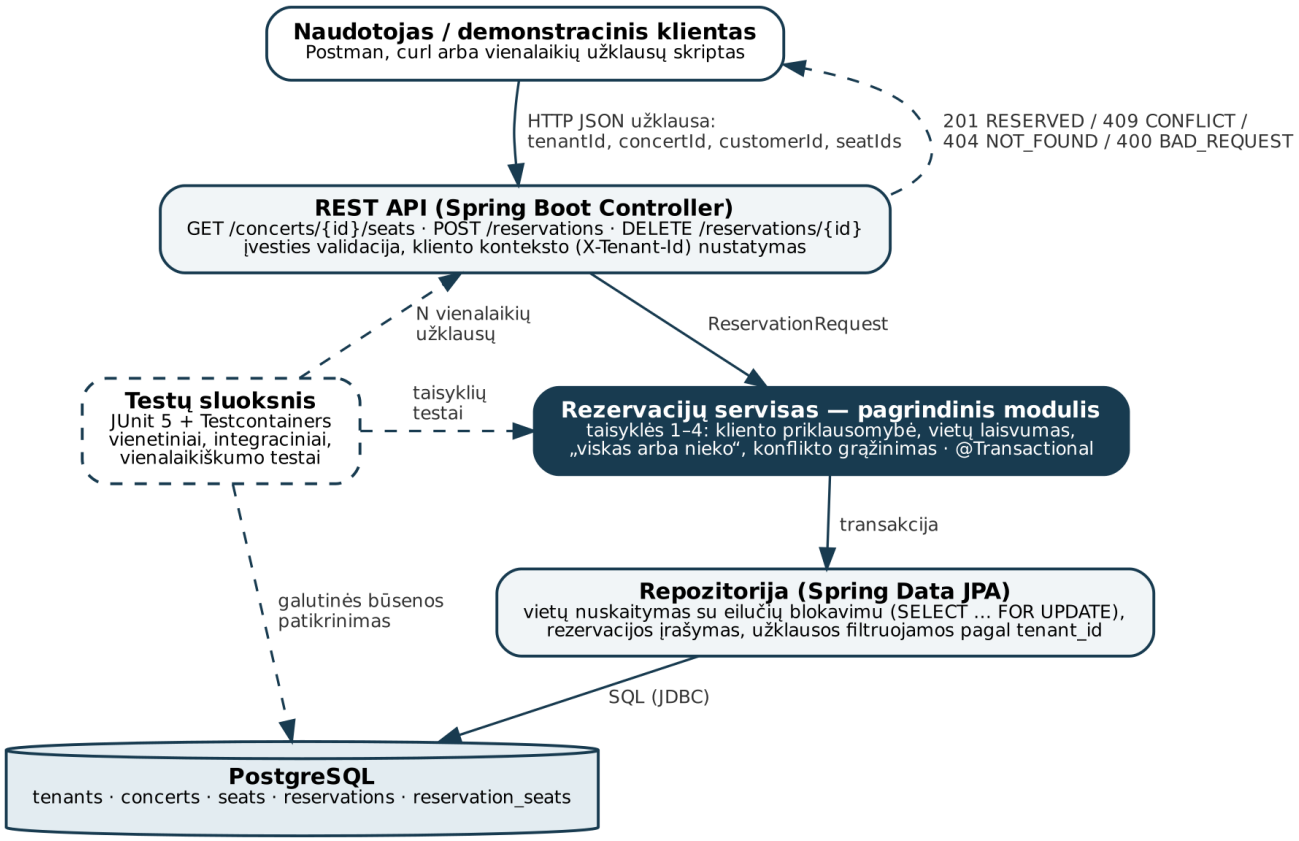
Numatytas projektavimo sprendimas: Rezervavimo operacija bus vykdoma duomenų bazės transakcijoje, naudojant tokį vienalaikių operacijų valdymo mechanizmą, kuris užtikrintų, kad tik viena operacija galėtų sėkmingai pakeisti tos pačios vietos būseną. Pradiniame variante planuojama naudoti transakciją ir duomenų bazės eilučių blokavimą arba kitą ekvivalentišką atominiu veiksmu pagrįstą sprendimą. Konkreti realizacija gali būti patikslinta įgyvendinimo metu.

Kaip patikrinsiu vėlesniame etape: Bus sukurtas automatinis integracinis testas, paleidžiantis daug vienalaikių rezervavimo užklausų. Papildomai bus atliekami pakartotiniai bandymai su skirtingu vienalaikių naudotojų skaičiumi ir tikrinama galutinė duomenų bazės būsena.

Sprendimo kaina arba ribojimas: Transakcijos ir blokavimas gali sumažinti lygiagretų užklausų apdorojimą, nes dalis operacijų turi laukti, kol kita operacija atlaisvins naudojamą duomenų bazės išteklių. Todėl sprendimas gali būti mažiau našus nei nesaugus variantas. Šiame projekte prioritetas teikiamas rezervacijų teisingumui, o našumo poveikis bus stebimas atliekant vėlesnius matavimus.

5. Pradinė sistemos struktūra

Paprasta schema



1 pav. ConcertSafe pradinė sistemos struktūra (pilnos rodyklės – užklausos kelias, punktyrinės – atsakymas ir testai)

Užklausos kelias: naudotojas (arba demonstracinis skriptas) siunčia HTTP JSON užklausą su kliento (tenant), koncerto, naudotojo ir vietų identifikatoriais. REST API sluoksnis patikrina įvesties formatą, nustato kliento kontekstą ir perduoda užklausą rezervacijų servisui. Servisas vienoje duomenų bazės transakcijoje per repozitoriją nuskaito pasirinktas vietas su eilučių blokavimu (SELECT … FOR UPDATE), pritaiko 3 skyriaus taisykles 1–4 ir arba įrašo rezervaciją bei pakeičia vietų būseną į RESERVED, arba atšaukia transakciją. Rezultatas grąžinamas naudotojui: 201 RESERVED, 409 CONFLICT, 404 NOT_FOUND (neegzistuojanti arba kitam klientui priklausanti vieta) arba 400 BAD_REQUEST (netinkama įvestis). Visi duomenys saugomi PostgreSQL; kiekviena lentelė turi tenant_id stulpelį, pagal kurį filtruojamos visos užklausos.

Sistemos dalis	Atsakomybė
REST API / Controller'is	Priima naudotojo užklausas ir grąžina rezultatus bei klaidų būsenas.
Rezervacijų servisas	Pagrindinė verslo logika: tikrina rezervavimo taisykles, valdo konfliktus ir inicijuoja saugią rezervavimo operaciją.
Repozitorija	Atsakinga už duomenų skaitymą ir pakeitimą duomenų bazėje.
Duomenų bazė	Saugo klientų, koncertų, vietų ir rezervacijų duomenis.
Testų sluoksnis	Tikrina pagrindines verslo taisykles, konfliktus ir klientų duomenų izoliavimą.

Planuojamos technologijos ir pasirinkimo priežastys:

• Java 21 – planuojama naudoti dėl objektinio programavimo galimybių, aiškios tipų sistemos ir tinkamumo struktūruotam verslo logikos projektavimui.

• Spring Boot – bus naudojama REST API ir sluoksniuotos sistemos struktūros realizavimui.

• PostgreSQL – pasirinkta reliacinė duomenų bazė, nes rezervavimo modulyje svarbios transakcijos, duomenų vientisumas ir vienalaikių operacijų kontrolė.

• JUnit 5 – automatiniams vienetiniams ir integraciniams testams.

• Testcontainers – integraciniai ir vienalaikiškumo testai vykdomi su tikra PostgreSQL versija Docker konteineryje, todėl rezultatai nepriklauso nuo lokaliai įdiegtos duomenų bazės konfigūracijos.

• Maven – projekto priklausomybių ir testų vykdymo valdymui.

6. AI panaudojimas

AI rengiant šį dokumentą

Taip, rengiant šį dokumentą naudojau AI. Naudojimas aprašytas lentelėje.

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
ChatGPT – projekto idėjos, struktūros ir formuluočių parengimas	Naudotos idėjos dėl rezervavimo modulio apimties, pagrindinio scenarijaus, verslo taisyklių, testų ir kokybės atributo.	Atmestos nereikalingos pilnos bilietų prekybos platformos funkcijos, pavyzdžiui, realūs mokėjimai, sudėtinga vartotojų sistema ir išorinės integracijos. Taip pat perrašytos formuluotės, kad jos tiesiogiai atitiktų pateiktus I dalies reikalavimus ir realistišką projekto apimtį.	Dokumento turinys sutikrintas su I dalies reikalavimais: pagrindinis scenarijus, testuojamos taisyklės, trys scenarijai, kokybės atributas, sistemos struktūra, rizikos ir tolesni darbai.

Planuojamas AI naudojimas kuriant sistemą

Kur ir kam naudosiu AI: AI planuojama naudoti kaip programavimo pagalbinę priemonę: kodo struktūros variantams, testų pavyzdžiams, SQL užklausų juodraščiams, dokumentacijos ir README formuluotėms bei galimoms klaidoms analizuoti.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Sugeneruotas kodas nebus laikomas teisingu vien dėl to, kad jį pateikė AI. Kiekvienas svarbesnis sprendimas bus peržiūrimas ir tikrinamas automatizuotais testais. Pats kodas bus ranka analizuojamas ir refaktorinamas pagal poreikį. Ypatingas dėmesys bus skiriamas rezervavimo transakcijoms, klientų izoliavimui ir vienalaikių užklausų rezultatams.

Ar AI bus sistemos funkcionalumo dalis: Ne.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
Duomenų modelio ir API projektavimas	Paruošta duomenų bazės schema, pagrindiniai objektai ir rezervavimo API kontraktas.	Aprašyti domeno objektus: klientą, koncertą, vietą ir rezervaciją. Sukurti duomenų bazės struktūrą ir pradinius duomenis. Įgyvendinti vietų peržiūros ir rezervavimo API.
Pagrindinio rezervavimo modulio realizavimas	Veikianti rezervavimo funkcija nuo REST API užklausos iki duomenų bazės atnaujinimo.	Įgyvendinti transakcinę rezervavimo logiką ir konfliktų valdymą.
Vienalaikių operacijų valdymo realizavimas	Įdiegtas transakcinis mechanizmas, apsaugantis nuo dvigubo tos pačios vietos rezervavimo.	Atlikti vienalaikių užklausų testą ir patikrinti galutinę duomenų būseną.

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
Automatinių testų ir klientų izoliavimo įgyvendinimas	Ne mažiau kaip 4 prasmingi testai, įskaitant vienalaikių rezervacijų ir tenant izoliavimo testus.	Parašyti automatinius funkcinius, konfliktinius, klaidos ir klientų izoliavimo testus.
README ir demonstracinio scenarijaus parengimas	Dokumentuotas paleidimas, testų vykdymas, pavyzdiniai duomenys ir prototipo demonstravimo seka.	Parengti README ir demonstracinį scenarijų.

Būsimo prototipo veikimo scenarijus: prototipo demonstracijoje bus parodyta visa rezervavimo eiga nuo HTTP užklausos iki galutinės duomenų bazės būsenos, daugiausia dėmesio skiriant vienalaikėms užklausoms. Demonstracija pradedama nuo švarių pradinių duomenų:

• tenant-a: koncertas concert-101 su 10 vietų A1–A10, visos būsenos AVAILABLE;

• tenant-b: koncertas concert-201 su vietomis B1–B5 (naudojamas klientų izoliavimui parodyti);

• naudotojai customer-1 … customer-100; visos užklausos žemiau siunčiamos į concert-101.

Demonstracijos žingsnis	Užklausa ir konkrečios vietos	Vienalaikių užklausų sk.	Laukiamas rezultatas
1. Vietų peržiūra	GET /concerts/concert-101/seats, X-Tenant-Id: tenant-a	1	Grąžinamos 10 vietų A1–A10, visos AVAILABLE.
2. Įprasta rezervacija	customer-25 rezervuoja [A1, A2]	1	**1 sėkminga**, 0 atmestų. 201 RESERVED; A1 ir A2 tampa RESERVED.
3. Konkurencija dėl vienos vietos	customer-1 … customer-100 vienu metu rezervuoja [A3]	100	**1 sėkminga, 99 atmestos** (409 CONFLICT). A3 turi lygiai vieną aktyvią rezervaciją.
4. Persidengiančios kelių vietų užklausos	50 užklausų rezervuoja [A4, A5], kitos 50 – [A5, A6]; visos paleidžiamos vienu metu	100	**1 sėkminga, 99 atmestos** (409 CONFLICT). A5 rezervuota; iš A4 ir A6 rezervuota tik laimėjusios užklausos vieta, kita lieka AVAILABLE – dalinių rezervacijų nėra.
5. Skirtingos vietos lygiagrečiai	customer-30 rezervuoja [A7], customer-31 – [A8] tuo pačiu metu	2	**2 sėkmingos**, 0 atmestų. Blokavimas neatmeta užklausų dėl skirtingų vietų.
6. Neegzistuojanti vieta	customer-40 rezervuoja [A9, A10, A11] (A11 neegzistuoja)	1	**0 sėkmingų, 1 atmesta** (404 „Seat A11 not found“). A9 ir A10 lieka AVAILABLE.
7. Kito kliento duomenys	X-Tenant-Id: tenant-b, customer-50 rezervuoja concert-101 vietą [A9]	1	**0 sėkmingų, 1 atmesta** (404, tenant-a duomenys neatskleidžiami). A9 lieka AVAILABLE.
Iš viso	205 rezervavimo užklausos (2–7 žingsniai)	—	**5 sėkmingos, 200 atmestų.** Galutinė būsena: 7 vietos RESERVED (A1, A2, A3, A5, A7, A8 ir A4 arba A6), 3 AVAILABLE (A9, A10 ir A4 arba A6). Nė viena vieta neturi daugiau nei vienos aktyvios rezervacijos.

Kaip bus vykdoma ir tikrinama: vienalaikės užklausos (3–5 žingsniai) bus paleidžiamos demonstraciniu Java skriptu / integraciniu testu su 100 gijų ExecutorService ir CountDownLatch, kad visos užklausos startuotų tuo pačiu momentu. Skriptas suskaičiuos 201 ir 409 atsakymus ir palygins juos su lentelėje nurodytais skaičiais. Po demonstracijos galutinė būsena tikrinama dviem būdais: GET /concerts/concert-

101/seats užklausa ir SQL užklausa, skaičiuojančia aktyvias rezervacijas kiekvienai vietai (laukiama: ne daugiau kaip 1). Visa seka bus pakartota 3 kartus nuo švarių pradinių duomenų – sėkmingų ir atmestų rezervacijų skaičiai kiekvieną kartą turi sutapti.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Neaišku, kuris vienalaikių operacijų valdymo mechanizmas bus tinkamiausias pasirinktoje technologinėje aplinkoje.	Pradžioje bus įgyvendintas paprastas transakcinis variantas ir atlikti vienalaikių užklausų testai. Prireikus bus palygintas alternatyvus optimistinio blokavimo sprendimas.
Vienalaikių testų rezultatai gali priklausyti nuo konkrečios lokaliai naudojamos duomenų bazės konfigūracijos.	Rezultatai bus tikrinami integraciniuose testuose su tiksliai aprašyta duomenų bazės versija ir nustatymais, o README bus nurodyta visa aplinka.
Per didelė projekto apimtis gali nukreipti dėmesį nuo pagrindinio modulio.	Bus ribojamas funkcionalumas iki rezervavimo, vietų peržiūros, atšaukimo ir klientų izoliavimo. Papildomos komercinės funkcijos nebus įtraukiamos.

Šaltiniai, jei naudojote

Šaltinių nenaudojau.